

PROGRAMMING II

TYPES AND OBJECTS

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

LANGUAGE TYPEDNESS



TYPES

Informal definition of **type**: a set of values together with operations on them

The **integer** type represents

- ▶ Values like -2 , -1 , 0 , 1 , and 2
- ▶ Operations like $+$, $-$, $*$, and $/$

☹ Theoretical computer scientists and logicians may disagree on that

TYPES (CONT'D)

In programming, types define a contract between variables and expressions

If a variable has type `integer`

- ▶ It must/may contain integer values
- ▶ It must/may be compared with expression yielding integer values
- ▶ It must/may be used in contexts where integer values are expected

The choice between must and may is up-to the language, unfortunately

WEAKLY AND STRONGLY TYPED LANGUAGES

Python is a **weakly typed** language

- ▶ A variable does not need a type declaration
- ▶ A variable may contain values belonging to different types

```
x = 100;  
x = "Hello_World";  
x = (1,2,3,4);
```

// who cares

Java is a **strongly typed** language

- ▶ A variable requires a type declaration
- ▶ A variable must contain only values belonging to the declared type

```
int x;  
x = 100;  
x = "Hello_World";
```

// syntax error

WEAKLY AND STRONGLY TYPED LANGUAGES (CONT'D)

Weakly typing

- ▶ Shorter programs, generally
- ▶ Programmer has more freedom, the language is more liberal in applying operations to values

Strongly typing

- ▶ Programmer has to be more disciplined
- ▶ More errors caught at compile-time

WEAKLY AND STRONGLY TYPED LANGUAGES (CONT'D)



PRIMITIVE TYPES

The Java `int` type

- ▶ Values are $-2^{31}, \dots, 2^{31} - 1$ and operations are $+, -, *, /, \%, -$

Here, $\%$ is the modulo operation: $b \% c$ is the remainder, when b is divided by c

- ▶ If b takes 67 and c takes 60, then $b \% c$ is 7

Here, $-$ is **overloaded**

- ▶ Can be the subtraction operation: $b - c$
- ▶ Can be the negation operation: $-b$

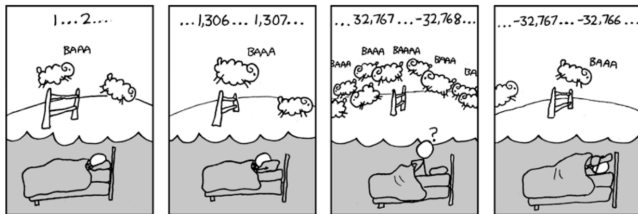
PRIMITIVE TYPES (CONT'D)

The Java `int` type

- Values are $-2^{31}, \dots, 2^{31} - 1$ and operations are $+, -, *, /, \%, -$

Java: operations of type `int` must produce a value of type `int`

- If a takes $2^{31} - 1$, then $a + 1$ is -2^{31}



PRIMITIVE TYPES (CONT'D)

The Java `int` type

- ▶ Values are $-2^{31}, \dots, 2^{31} - 1$ and operations are $+, -, *, /, \%, -$

The Java `double` type

- ▶ Values are like 2.51×10^{-6} or 24.9, and operations are $+, -, *, /, \%, -$

The Java `char` type

- ▶ Values are like 'V', '?', or '\n', and no operations

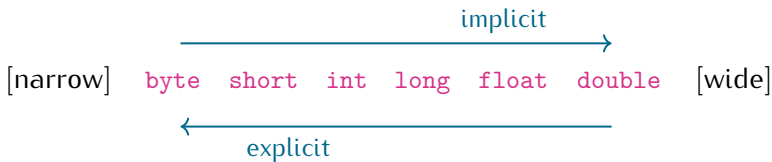
The Java `boolean` type

- ▶ Values are *true* or *false*, and operations are `!, &&, ||` *// not, and, or*

TYPE CONVERSION

Implicit cast: `float f = 3;` converts the `int 3` to the `float 3.0`

Explicit cast: `int i = (int) 3.2;` converts the `double 3.2` to the `int 3`



CHARACTERS ARE NUMBERS

Actually, the `char` type contains single-byte integers

- ▶ `(int) 'V'`: Unicode representation in decimal, that is, 86 *// cast not needed*
- ▶ `(char) 86`: decimal representation in Unicode, that is, 'V'

Characters encode Unicode as 16-bit hexadecimal literals

- ▶ `'\u0041'` is 'A', `'\u0056'` is 'V'
- ▶ `'\u0034'` is '\$', `'\u00A9'` is '©'

NON-PRIMITIVE TYPES

In Java, everything else

- ▶ Everything is an object
- ▶ Except primitive types

// Java is a nearly pure OO language

The Java String type

- ▶ Values are like “hello” or “world”, and operations are user-defined
- ▶ Java makes available a special **concatenation** operation $+$ on string objects
- ▶ If a takes “Hello ” and b takes “World”, then $a + b$ is “Hello World”

JAVA OBJECTS



MEMORY IN JAVA

An objects represents a sandboxed memory area

- ▶ Such area is accessible via message passing only
- ▶ Fields are subsets of memory locations
- ▶ Methods are the way we can interact with such memory locations

In Java

- ▶ Everything is an object, thus there are no global variables (unlike C)
- ▶ Programmers cannot manually allocate/deallocate memory (unlike C)
- ▶ Process memory is automatically optimized by the Java [Garbage Collector](#)

MEMORY IN JAVA (CONT'D)



POINTERS IN JAVA

Java does have pointers, but there is no pointer arithmetic

- ▶ Memory management is very constrained, to mitigate programmer errors

Java actually uses pointers all the time

- ▶ To reference objects
- ▶ To pass method arguments (by value)

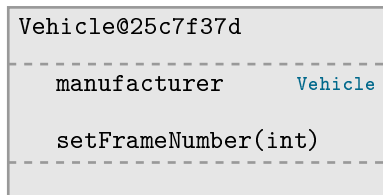
Technically speaking, every non-primitive variable in Java has type reference

- ▶ A variable of type `String` will contain a pointer of a `String` object

OBJECTS IN JAVA

An **object reference** contains a class name and a memory address

```
Vehicle myCar;  
// myCar contains Vehicle@null  
  
myCar = new Vehicle();  
// myCar contains Vehicle@25c7f37d
```



Messages to an object are sent by using its reference (with dot-notation)

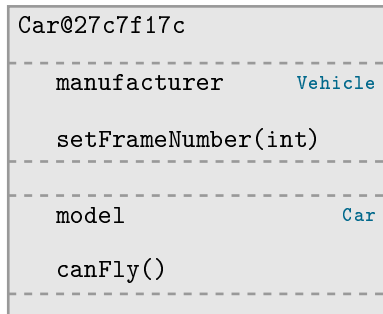
- ▶ `myCar.setFrameNumber(2)`
- ▶ The message is sent to the memory area 25c7f37d

OBJECTS IN JAVA (CONT'D)

An **object reference** contains a class name and a memory address

```
Vehicle myCar;  
// myCar contains Vehicle@null  
  
myCar = new Car();  
// myCar contains Car@27c7f17c
```

A Car objects **inherits** all fields and methods of Vehicle objects



MEMORY ERRORS IN JAVA

Java is yet not fully memory-safe

```
Vehicle myCar1 = new Vehicle();  
// myCar1 contains Vehicle@37c7a17c
```

```
Vehicle myCar2;  
// myCar2 contains Vehicle@null
```

Calling `myCar2.setFrameNumber(5)` yields
a `NullPointerException` error!

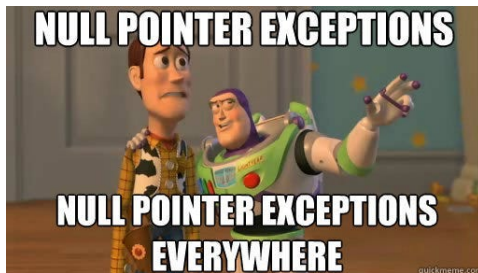
Vehicle@37c7a17c

manufacturer Vehicle

setFrameNumber(int)

Vehicle@null

MEMORY ERRORS IN JAVA (CONT'D)



The special value `null` can be used by programmers

- ▶ Into conditions: `myCar2 == null`, to check null-pointer exceptions
- ▶ Into assignments: `myCar1 = null`;

// Why?

Q + A

